CampusKit

GOOD TO GO.

External Design Document v1

Campus equipment lending application

COMP229 - Web Application Development
Project Part 1 | First release | 9 November 2026

DRAFT FOR TEAM REVIEW

Proposed team name: CampusKit
Members and student IDs: [Complete with actual 4-6 members]
Repository URL: [Add actual team repository]
Board URL: [Add actual team task board]
Demo URL: [Add after recording]

Contents

Section	Page
Purpose, scope and user requirements	2
Architecture, collections and access rules	3
Student wireframes	4
Staff and account wireframes	5
Initial screenshot: sign-in	6
Initial screenshot: registration	7
Verification, limitations and completion work	8

01 / Purpose and scope

CampusKit helps students borrow individually tracked campus equipment for creative and academic projects. Staff maintain the inventory, review requests, and record equipment returns. The application is a proposed classroom project; it does not claim to be a deployed college service.

First-release requirements

User	Requirement / expected outcome
Student	Register and log in; browse equipment; search and filter by category or availability.
Student	Request an available asset with a purpose and valid due date; view, edit or cancel pending requests; delete requests without active loans.
Administrator	Create, read, edit and delete eligible equipment records; inspect all borrowing requests.
Administrator	Approve or reject pending requests; record physical returns; manage eligible student accounts.
Both	Read/update own profile; change password with current-password verification.
System	Hash passwords; verify JWTs; enforce roles/ownership; persist records in MongoDB and return useful validation errors.

Boundaries

One record means one physical asset. A pending request does not reserve equipment; approval does. Return dates can be today through the next 30 days. This release uses immediate lending rather than calendar-based reservations. Email recovery, fines, renewal rules, uploads and institutional single sign-on are outside Part 1.

Acceptance example

A student requests CAM-001. Staff approve it; the request becomes approved and CAM-001 becomes on-loan. A competing approval for CAM-001 fails. Staff record its return; the request becomes returned and CAM-001 becomes available.

02 / Architecture and data

React provides the view. Express routes apply authentication and authorization middleware, then call controllers. Controllers enforce business rules and use Mongoose models to read/write MongoDB. Development uses Vite on port 5001 and the API on port 5002; the Vite proxy forwards /api calls.

Layer	Files / responsibility
View	client/src/App.jsx and styles.css: screens, forms and API requests.
Controller	server/controllers/: auth, users, equipment and borrowing logic.
Model	server/models/: User, Equipment and BorrowRequest schemas.
Routing / security	server/routes/api.js and middleware/: JWT, roles, validation and errors.

Collections and relationships

Collection	Principal fields and constraints
users	name, unique email, hashed password, role, tokenVersion. Registration creates students only.
equipment	name, category, unique assetTag, description, location, status, activeRequest reference.
borrowrequests	user and equipment references, snapshot names/tag, purpose, dueDate, status, decisionNote. Partial unique index prevents duplicate pending/approved requests for one user/asset.

Relationships: one user has many requests; one asset has many historical requests and at most one approved loan. activeRequest identifies the asset's current approved request.

Access and states

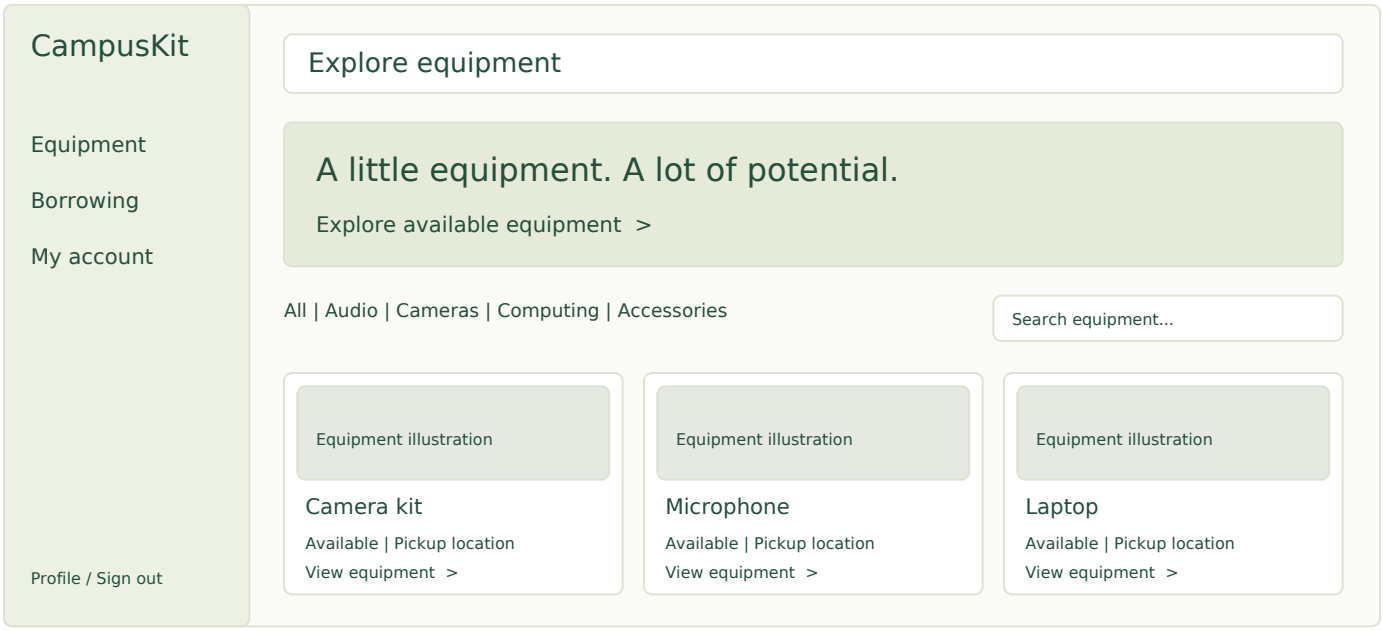
Students can access their own requests/profile. Administrators manage inventory and lending decisions. User Create is registration; Read/Update/Delete use /users endpoints. Equipment and requests expose full CRUD under /api/equipment and /api/requests. Full endpoint details are in API_REFERENCE.md.

Request transitions: pending to approved, rejected, cancelled or deleted; approved to returned; returned/rejected/cancelled to deleted by the owner. Equipment transitions: available to on-loan on approval; on-loan to available on return; staff can toggle available/maintenance when no loan is active.

03 / Student wireframes

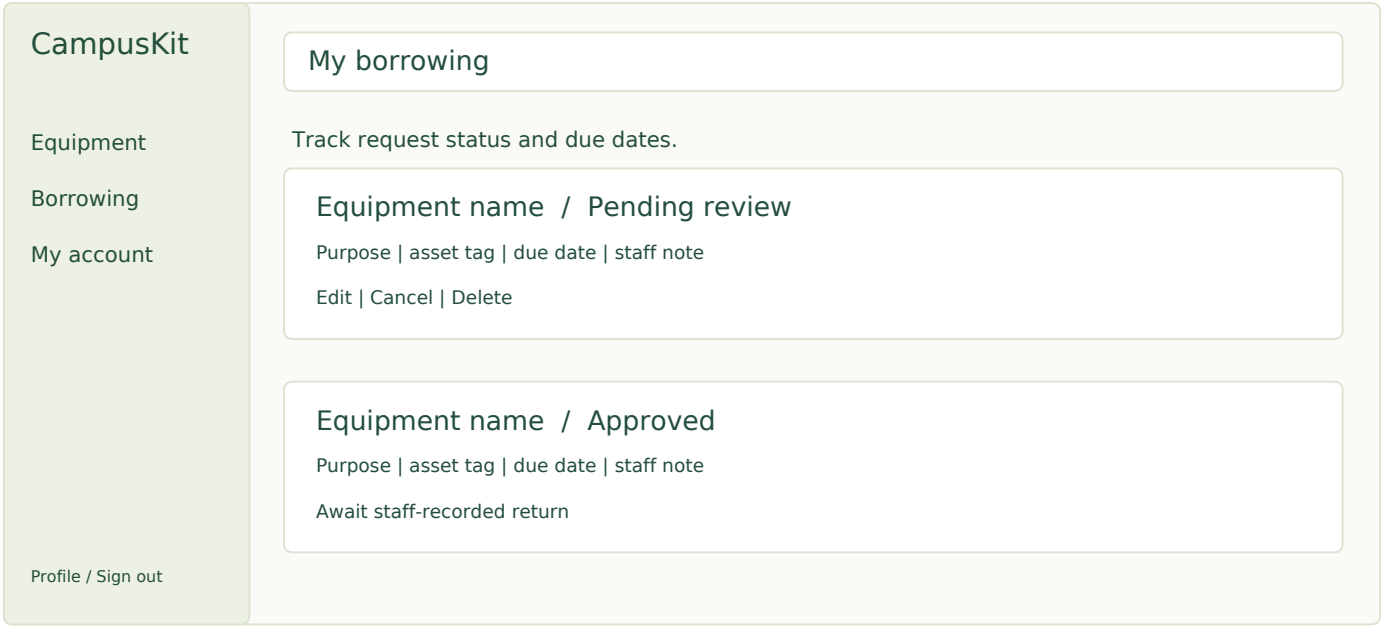
Low-fidelity layout plans. These show navigation, information placement and primary actions; they are not screenshots of executed workflows.

Equipment collection



Browse categories, search, show available assets, inspect equipment and open the request form. Equipment details include pickup location and an individual asset tag.

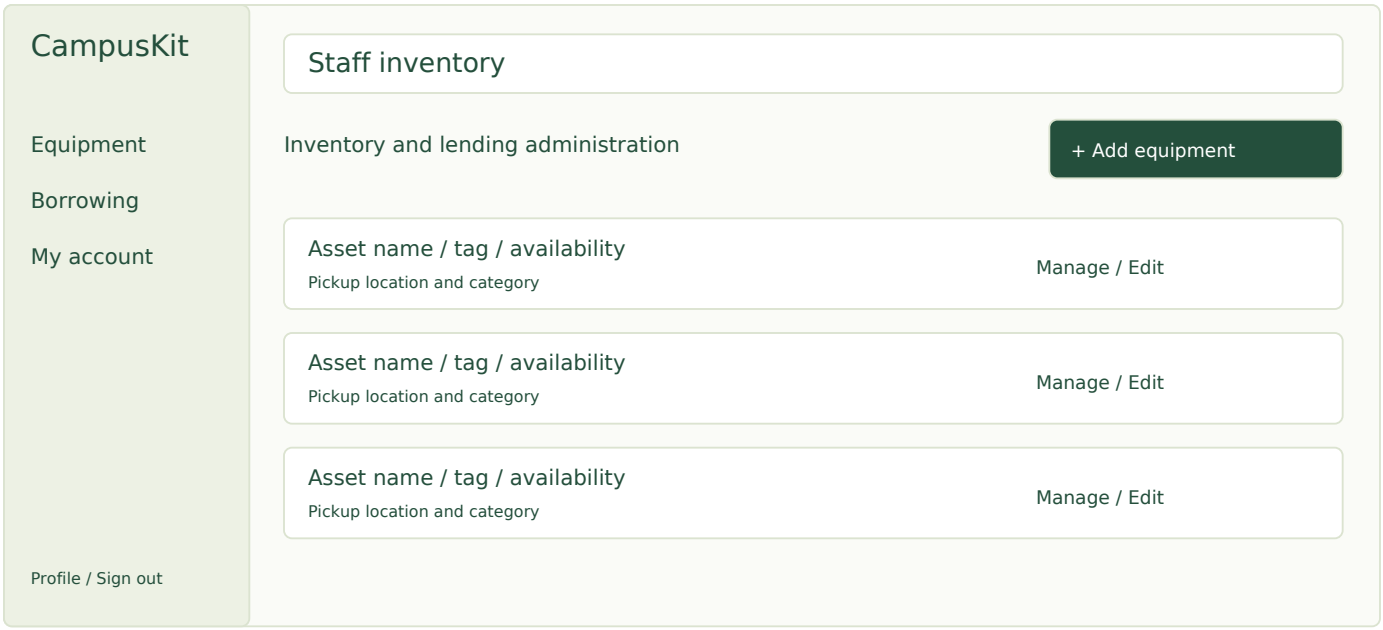
My borrowing



Each request displays its status, purpose, due date and staff note. Only allowed actions appear; server permissions remain authoritative.

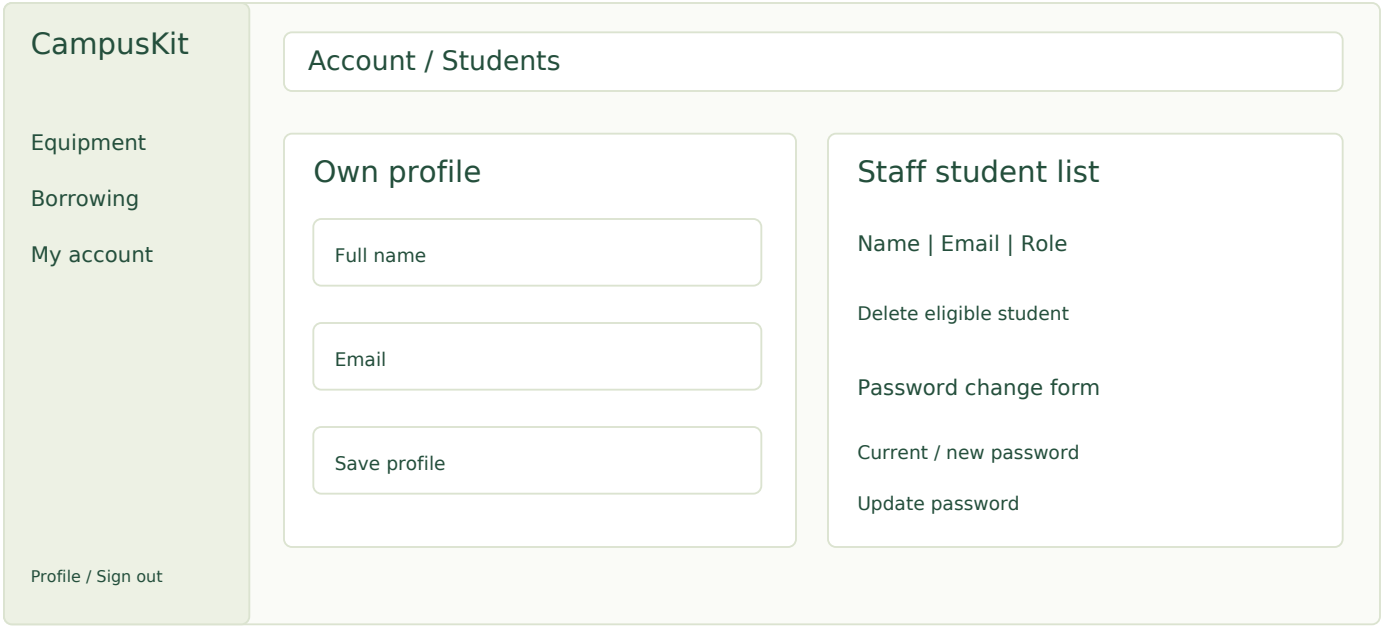
04 / Staff and account wireframes

Inventory administration



Add/manage opens a form for name, category, asset tag, location, description and availability. Active-loan availability changes must use the return workflow. A confirmation precedes permanent deletion.

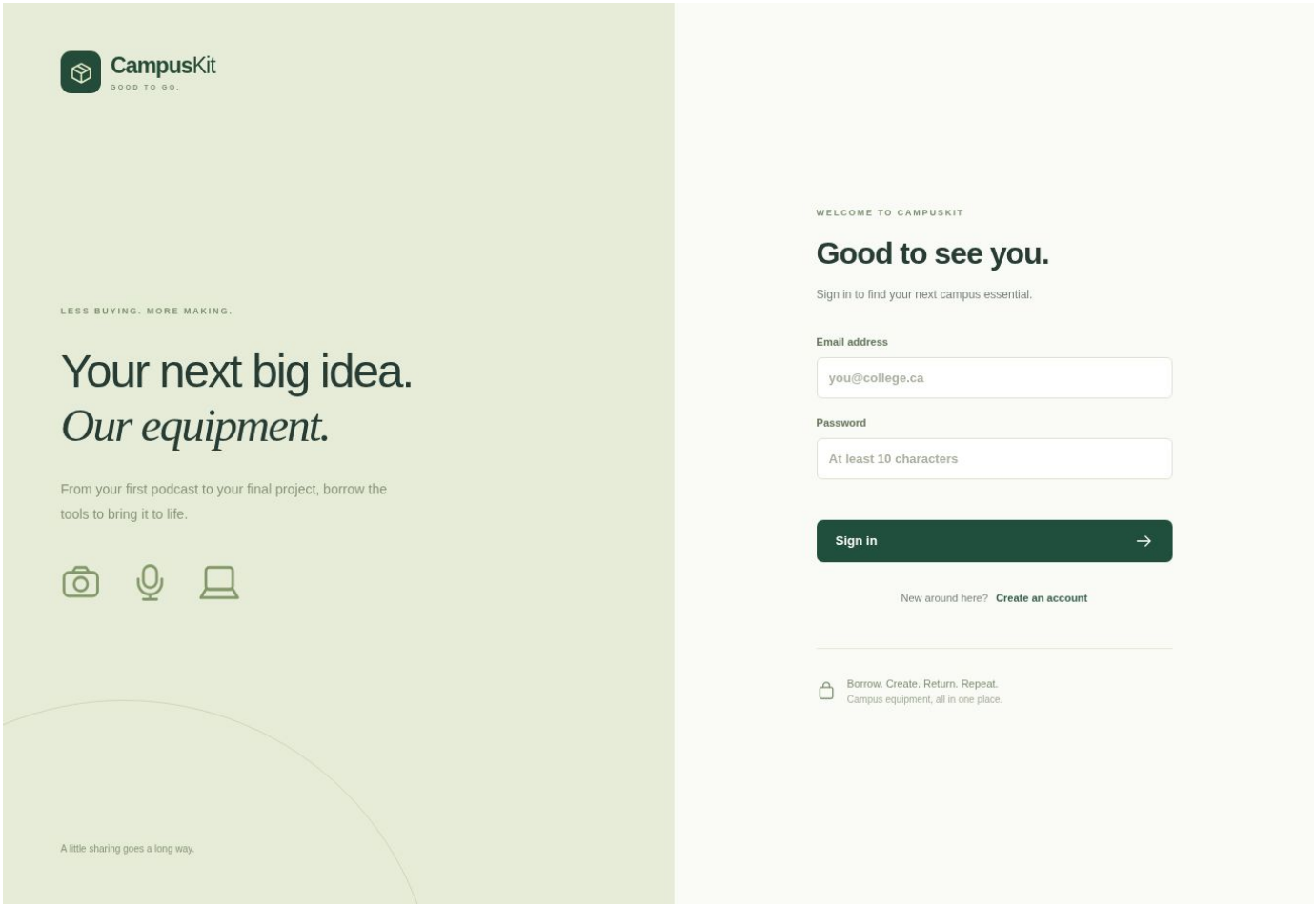
Account and student management



Own profile includes name/email and a separate password form. Staff can inspect the student list and delete accounts without active/pending requests. Administrator accounts are retained. Borrowing requests use the request-card layout with Approve, Reject or Record return actions.

05 / Initial screenshot - sign-in

Captured from the built React frontend during layout inspection. The account screen is implemented. This static preview does not demonstrate a live MongoDB-backed login.



Design notes

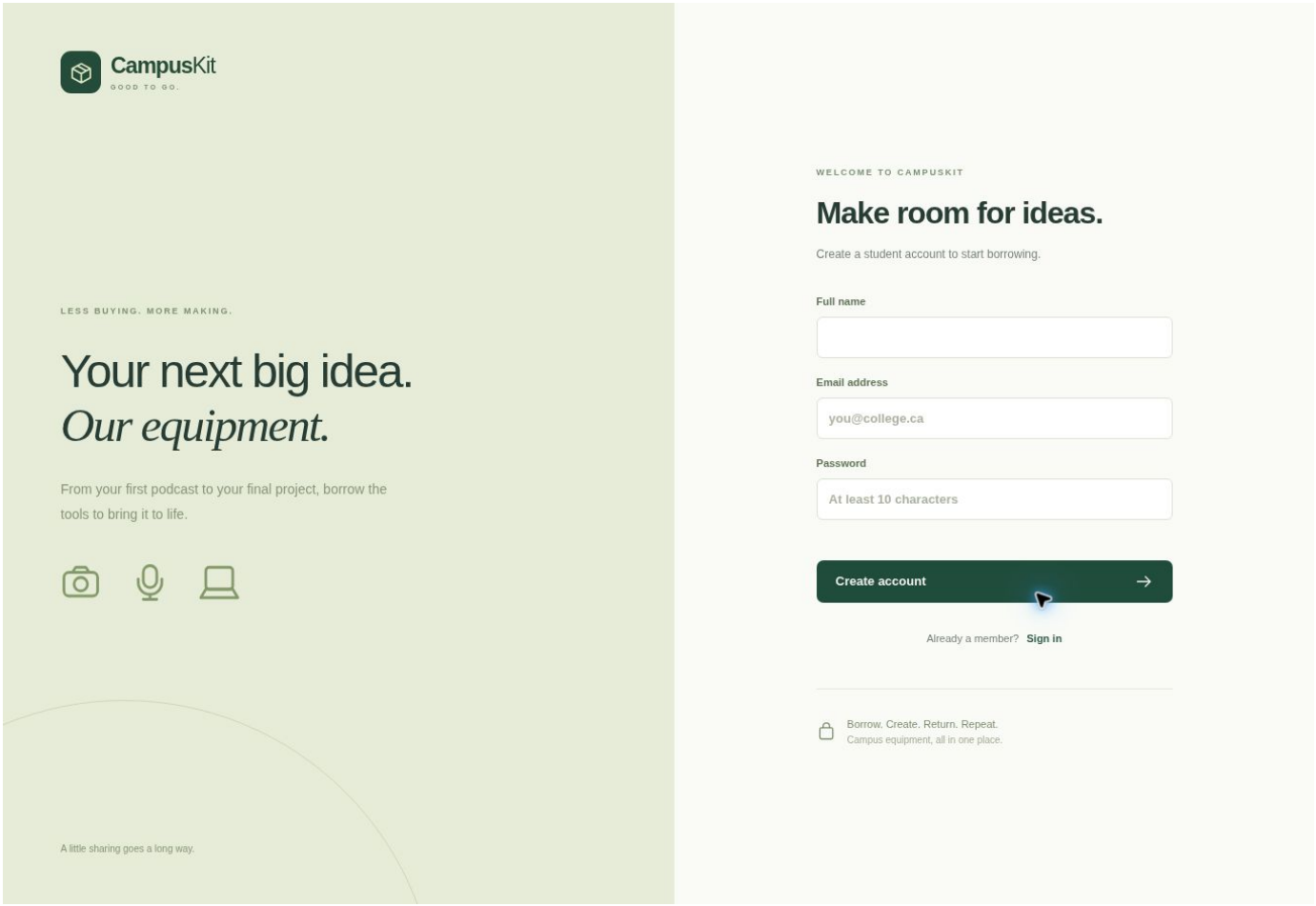
The split layout introduces the lending concept while keeping the sign-in form prominent. The green/sage palette and simple equipment illustrations are consistent across the application. Form fields have explicit labels and native keyboard navigation. The CSS switches to a single-column layout on narrow screens; a mobile browser capture is still required from the team.

Entry actions

Students can switch to registration or sign in. Seeded administrator accounts use the same login form. Tokens stay in memory, so a page refresh requires signing in again.

06 / Initial screenshot - registration

Captured after switching the implemented account interface to student registration. No real student details were entered and no account was created in this screenshot.



Validation and security

Registration collects name, email and password. The server creates only student accounts, ignores client role input, validates fields, hashes the password and rejects duplicate email addresses. Passwords must contain at least 10 characters and at most 72 UTF-8 bytes.

Screenshots to add after team testing

Capture the authenticated equipment collection, student borrowing list, administrator inventory form, approval/return flow and representative Postman results. Keep credentials and tokens out of screenshots. Replace draft identity fields before submission.

07 / Verification and next steps

Verification performed on this package

Check	Result and practical limit
React build	npm run build passed. This verifies compilation, not every browser interaction.
API integration suite	12 tests passed against a real disposable MongoDB instance; separate from the persistent app database.
Covered API behavior	Login, password hashing, role/ownership controls, equipment/request CRUD, duplicates, dates, active-loan guards, competing approvals and token invalidation.
Browser layout review	Sign-in/registration screens reviewed in a static preview. Full authenticated end-to-end and mobile browser QA remain for local testing.
Hosted preview limit	The hosted preview could not start its temporary MongoDB process. This is not evidence that authenticated browser flows passed.
Postman collection	Executed with Newman: 31 requests, 34 assertions, zero failures, on an isolated test database.

Known implementation limits

Cross-collection updates are not one transaction. The approval flow atomically claims the asset, conditionally updates the request and compensates if the request changed. An abrupt failure between writes can require reconciliation. Cross-collection deletion checks are also not isolated. Production follow-up should use replica-set transactions and audit/retention rules. Pagination and token refresh are not included.

Required team completion

Confirm 4-6 members and roles; review/complete/sign the contract; confirm the logo; create the real GitHub repository with each member's genuine commits; create and use an actual task board; export its current backlog PDF; complete the two-slide deck with actual IDs/photos; capture authenticated screens/API evidence; record and upload a 5-10 minute demo; insert real URLs and remove draft labels after review.

References

Course source: Project Part 1.docx (provided). Technical references: Express 5 error handling; Mongoose 8 findOneAndUpdate; MongoDB unique indexes. Full official documentation links are in README.md.